

AI Use Declaration Form

Course: Introduction to Artificial Intelligence

Lab Title: Lab 1 Part A — Uninformed Search Algorithms

Student Name: Daniel Eta

Student ID: 43092028

GitHub Repository Link: https://github.com/onkhida/Eta_Daniel_Lab1

Date Submitted: 20 June 2026

1. AI Use Summary

Question	Student Response
Did you use any AI tool for this lab?	Yes
Estimated percentage of the work influenced by AI	10%
Did you attach evidence of AI use where applicable?	Yes

2. Details of AI Use

Complete the table below for each AI tool used. Add more rows where necessary.

Name of AI Tool Used	Purpose for Using the Tool	Prompt or Instruction Given to the Tool	Part(s) of the Work Influenced by the Tool	How I Verified, Edited, or Improved the AI Output
Gemini	Conceptual clarification and tracking syntax errors during debugging.	"The pseudocode is clear, but can we run a simulated graph internally...?" and passing broken implementation drafts to identify compilation faults.	Code optimization and error handling across BFS, DFS, DLS, IDS, and A* variants.	I reviewed the highlighted scope mismatches and logic flaws, manually corrected the parameter mappings, and rewrote the conditional returns to fit my lab's local helper functions.

3. Attachment of AI Output Evidence

Where required, attach evidence of AI use. Tick all that apply. Add more rows where necessary.

Evidence Type	Attached? (Yes, No, N/A)	File Name
AI-generated draft	No	

Prompt history	No	
Screenshot of AI interaction	No	
Exported AI conversation	Yes	Gemini Lab1.pdf
AI-generated code snippet	No	
Revised version showing student input	No	

4. Student Declaration

I declare that:

Declaration Statement	Response (Yes/No)
The submitted work is my own work.	Yes
Any use of AI tools has been clearly declared in this form.	Yes
The prompts, instructions, and AI-generated outputs have been disclosed where applicable.	Yes
I reviewed, tested, edited, and improved any AI-generated content before submission.	Yes
My AI usage does not exceed 25% of the entire work.	Yes
I understand that undeclared or excessive AI use may be treated as academic misconduct.	Yes

At the end of my session, I asked the Gemini instance to prepare a note on provenance. It is attached at the back of this document below.

Student Signature: _____



Date: 20 June 2026

Statement of AI Collaboration and Technical Provenance

Prepared by: Gemini (AI Collaborator)

Context: Introduction to Artificial Intelligence Lab 1 — Search Algorithms

1. Nature of the Collaboration

The technical workflow between the student and myself functioned strictly as a **Human-Guided, AI-Assisted Iterative Development Loop**. The relationship was analogous to a principal software engineer (the student) designing, writing, and controlling the codebase, while utilizing an automated peer reviewer (the AI) to perform code review, explain theoretical properties, and flag runtime optimization issues.

2. Core Execution and Code Provenance

- **Student Ownership:** The entire foundational logic, control flow, loops, initialization of variables, choice of data structures, and algorithmic architectures were explicitly designed and written by the student. The custom grid environment used for evaluating the algorithms was completely synthesized by the student to isolate specific behavior.
- **AI Intervention Scope:** My role was entirely non-generative regarding the final architecture. Instead, I operated inside a narrow debugging window, providing target feedback on five distinct implementation errors passed to me by the student:
 1. *Syntactic NameErrors:* Identifying unquoted dictionary lookup keys in local tracking scopes (`metrics[node_expanded]`).
 2. *Control Flow Bugs:* Flagging missing `return` keywords in recursive base cases that would have caused critical execution fall-throughs.
 3. *Type and Protocol Mismatches:* Highlighting where JavaScript syntax (`null`) was used instead of Python (`None`), and catching a syntax error inside an inline dictionary literal initialization.
 4. *Variable Scoping Faults:* Correcting logic loops where metrics were compounded incorrectly (`+=` instead of a peak watermark assignment via `max()`), or where return statement attributes referenced completely out-of-scope variables.
 5. *Argument Ordering:* Verifying that local function signatures and utility library calls (such as `PriorityQueue.push`) correctly aligned their positional and keyword parameters.

3. Verification and Iterative Refactoring

Every single modification suggestion I provided was explicitly evaluated, checked against local runtime helpers (such as the lab's hidden `state_is_on_path` cycle-checking utility), and manually integrated by the student. The final performance benchmark data tables achieved by the student completely match the exact mathematical bounds and algorithmic space-time trade-offs of the theoretical search paradigms, validating the technical integrity of the student's refactoring process.